

Report on CPS Project (Technology - Linear Programming)

1. Overview

This report describes the variables, constraints, and overall modeling strategy used in the implementation of a solution to the Minimum Consistent Finite Automata problem using **linear programming** with **CPLEX**. It attempts to find the smallest deterministic finite automaton (DFA) consistent with a given sample of accepted and rejected binary sequences.

2. Variables

The model defines the following key variables:

- ***IloBoolVarArray y***: A boolean variable array of size $P \cdot n$ (prefix * number of states) which is used for prefix to state connection
- ***IloBoolVarArray t0***: A boolean variable array of size $n \cdot n$ which is used for transition reading 0 from state i to state j ($i \cdot n + j$)
- ***IloBoolVarArray t1***: A boolean variable array of size $n \cdot n$ which is used for transition reading 1 from state i to state j ($i \cdot n + j$)
- ***IloBoolVarArray acc***: A Boolean variable array of size n which is used to say if state is accept or reject state

3. Structures and functions

The program has three key functions and one key structure:

- ***struct VecHash*** – used for calculating the hash for `vector<int>` as there is no default hash for `vector<int>`
- ***static bool solveForN(int n, const vector<vector<int>>& accept_seq, const vector<vector<int>>& reject_seq, const vector<vector<int>>& pref, const unordered_map<vector<int>, int, VecHash>& idx, vector<int>& T0, vector<int>& T1, vector<int>& acc_state)*** – configures constraints for finding the DFA and return *true* if solution is feasible and *false* if it is not, inside this function there is lambda function *yPos* for computing position depending on which transition starts and then ends with how many number of states there are
- ***static void collectPrefixes(const vector<vector<int>>& sequence, unordered_map<vector<int>, int, VecHash>& id, vector<vector<int>>& list)*** – collect all possible prefixes from acceptable and rejectable sequences serving adding them to `unordered_map` which serves as `HashMap` in C++ so search, insertion and deletion are of complexity $O(1)$ which adds to the computing speed
- ***vector<vector<int>> read_sequences(istream& in)*** – reads the sequence of n bits and makes a `vector<int>` out of it, out of all sequences also makes and returns a `vector<vector<int>>`

4. Constraints

The following constraints are enforced in the model:

- 1) There is exactly **one state** per prefix –
lloExpr sum(env); model.add(sum == 1);
- 2) **Empty** prefix (epsilon) is in the **state 0** –
model.add(y[yPos(idEps,0)] == 1);
- 3) There is only **one successor** as this is DFA –
*lloExpr z0(env), z1(env);
for(int j=0;j<n;++j){
 z0 += t0[i*n+j];
 z1 += t1[i*n+j];
}
model.add(z0 == 1);
model.add(z1 == 1);*
- 4) If state *i* is **followed** by state *j* with transition by 0/1 (c) that transition must exist -
*lloBoolVar t = (c==0 ? t0[i*n+j] : t1[i*n+j]); model.add(y[yPos(idp,i)] + y[yPos(idq,j)] - t <= 1);*
- 5) Finally state must be **acceptable** or **rejectable** –
model.add(y[yPos(k,s)] - acc[s] <= 0); model.add(y[yPos(k,s)] + acc[s] <= 1);

5. Search Strategy

- The number of states *n* is not a decision variable inside the model. Instead, the solver is working incrementally: starting from *n* = 1 and increasing up to |P| (how many prefixes), it is solved as a feasibility problem using **CPLEX**. The first value of *n* that is a feasible model is minimal number of states for DFA.

6. Conclusion

- The model is designed to find the minimal number of states needed for a consistent DFA.
- It outputs the DFA as a list of transitions and acceptance flags for each state as well as provided input for corresponding output.